



Санкт-Петербургский государственный университет
Кафедра системного программирования

Экспериментальное исследование алгоритмов для регулярных запросов

Максим Александрович Родионов, группа 23.Б15-мм

Санкт-Петербург
2025

Постановка задачи

В данной работе исследуется производительность решения задачи достижимости с регулярными ограничениями¹ на ориентированных графах двух алгоритмов на основе достижимости:

- между всеми парами вершин (`tensor_based_rpq`)
- для заданного множества стартовых вершин (`ms_bfs_based_rpq`)

Исследование направлено на ответы на следующие вопросы:

- Какое представление разреженных матриц и векторов лучше подходит для каждой из решаемых задач?
- Начиная с какого размера стартового множества выгоднее решать задачу для всех пар и выбирать нужные?

¹Regular Path Querying, RPQ

Описание набора данных

- Графы (из CFPQ_Data):
 - ▶ biomedical (341 вершин, 459 рёбер)
 - ▶ wine (733 вершин, 1839 рёбер)
 - ▶ pizza (671 вершин, 1980 рёбер)
 - ▶ core (1323 вершин, 2752 рёбер)
- Запросы:
 - ▶ $(m_1|m_2) * m_3$
 - ▶ $(m_3|m_4) + m_1*$
 - ▶ $(m_1|m_2|m_3|m_4) * m_1$
 - ▶ $m_3(m_1|m_2|m_3|m_4)*$
- Наиболее встречающиеся метки:

Граф	m_1	m_2	m_3	m_4
biomedical	type	label	subClassOf	comment
wine	type	rest	first	onProperty
pizza	disjointWith	type	subClassOf	onProperty
core	type	isDefinedBy	label	comment

Определение стартовых вершин

- Количество стартовых вершин:
 - ▶ **Абсолютные:** 1, 5, 10 вершин
 - ▶ **Относительные:** 5%, 10%, 20%, 35%, 50%, 70%, 100%
- Процентные значения рассчитывались от общего числа вершин в графе
- Количество достижимых пар для каждого графа и регулярного выражения:

Граф	Regex 1	Regex 2	Regex 3	Regex 4
biomedical	122	671	130	1181
wine	1221	1805	1631	869
pizza	1816	1516	721	1368
core	715	2597	1156	269

Описание эксперимента

Для вопроса №1:

- Форматы разреженных матриц (из `scipy.sparse`):
 - ▶ `csr_array` — Compressed Sparse Row
 - ▶ `csc_array` — Compressed Sparse Column
 - ▶ `dok_array` — Dictionary Of Keys
 - ▶ `lil_array` — Row-based List of Lists
- Каждая конфигурация: тип матрицы, граф и регулярное выражение

Для вопроса №2:

- Каждая конфигурация: регулярное выражение, граф и размер стартового множества

Технические детали:

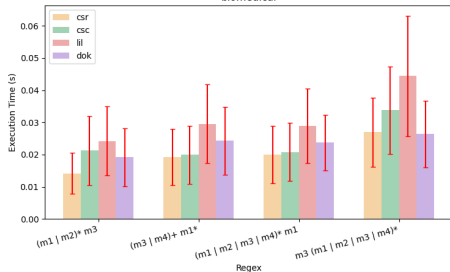
- Арифметические операции с матрицами: библиотека `SciPy`
- Для каждой конфигурации оба алгоритма запускались по 20 раз
- По результатам рассчитаны: среднее время и 95% доверительный интервал

- Процессор: Intel Core i7-11370H с зафиксированной частотой 3.3 GHz
(**sudo cpupower frequency-set -min 3300MHz -max 3300MHz**)
- ОЗУ: 16ГБ
- ОС: Linux Kubuntu 24.04
- Версия Python: 3.12.5
- Настройки производительности: использовался режим performance (**sudo cpupower frequency-set -g performance**)

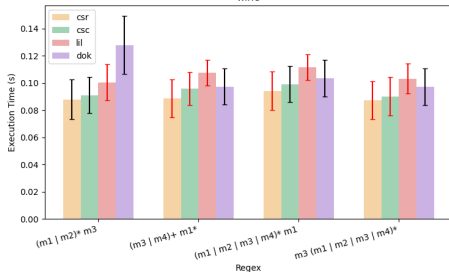
Результаты по вопросу №1 для tensor_based_rpq

Performance of tensor_based_rpq (Start Vertices: 50% of reachable)

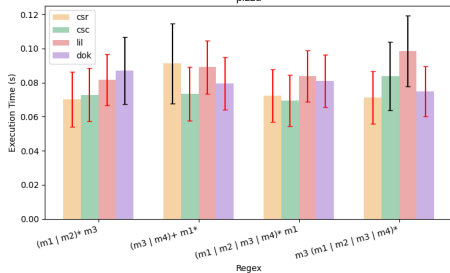
biomedical



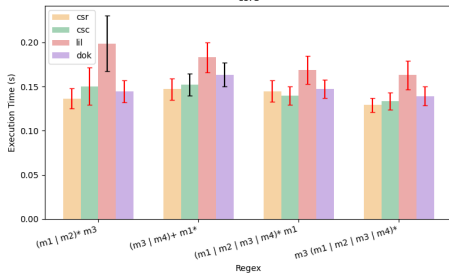
wine



pizza



core



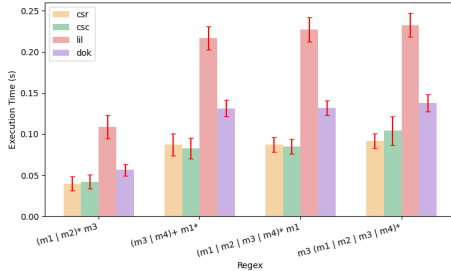
Вывод для tensor_based_rpq

- Сравнение производительности:
 - ▶ **CSR** — наиболее быстрый формат
 - ▶ **CSC** — практически идентичен **CSR** по времени, иногда чуть медленнее
 - ▶ **LIL** — медленнее всех остальных
 - ▶ **DOK** — также медленнее, чем **CSR/CSC**, но быстрее **LIL**
- Обоснование:
 - ▶ Алгоритм включает построение тензорного произведения и возведение матрицы в степень, что требует эффективного умножения и сложения разреженных матриц, для чего меньше подходит **LIL** и **DOK**, при этом **CSR** обеспечивает более быстрый доступ по строкам, что необходимо при построении результата

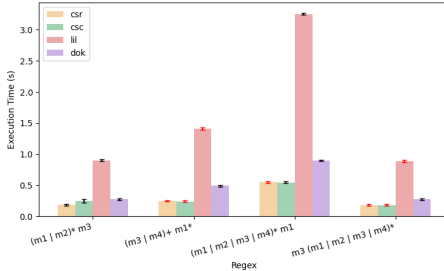
Результаты по вопросу №1 для ms_bfs_based_rpq (1/2)

Performance of ms_bfs_based_rpq (Start Vertices: 10% of reachable)

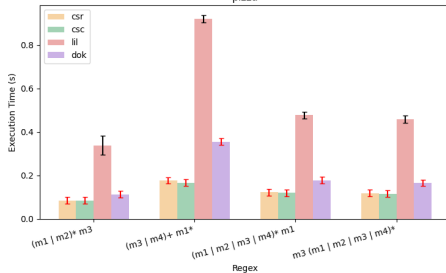
biomedical



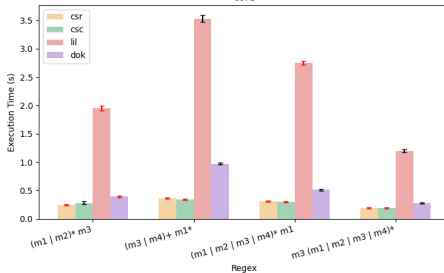
wine



pizza

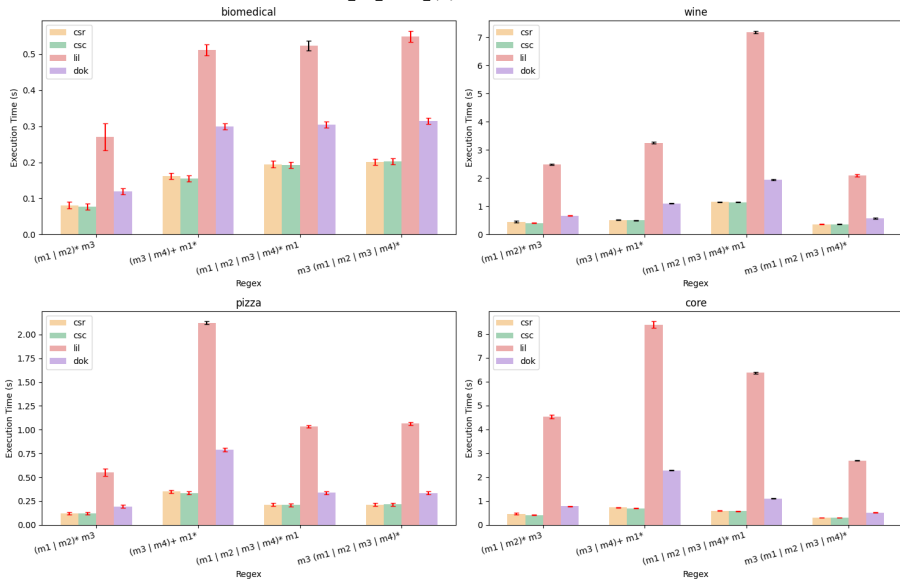


core



Результаты по вопросу №1 для ms_bfs_based_rpq (2/2)

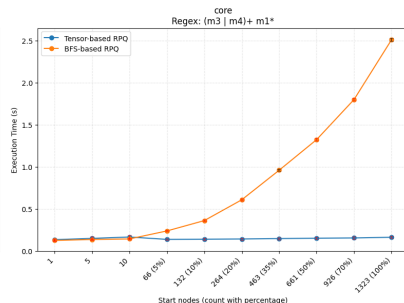
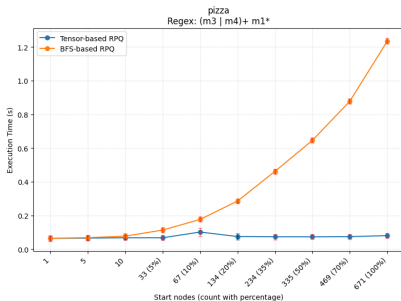
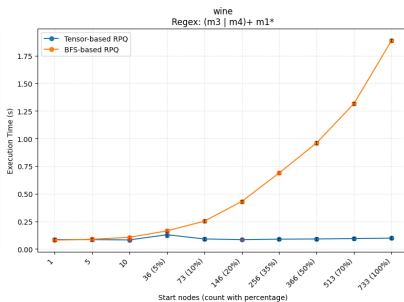
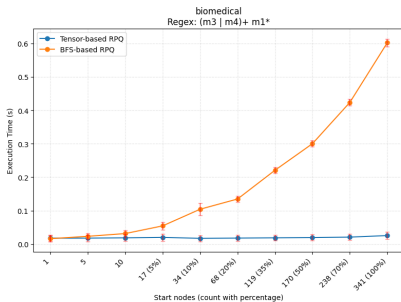
Performance of ms_bfs_based_rpq (Start Vertices: 25% of reachable)



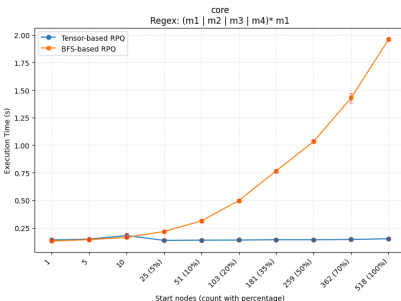
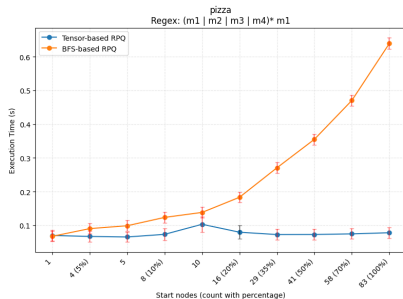
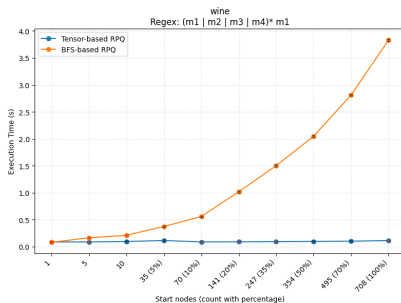
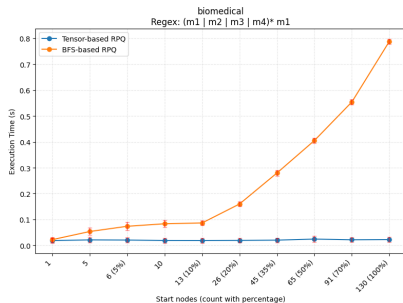
Вывод для `ms_bfs_based_rpq`

- Сравнение производительности:
 - ▶ **CSC** — наиболее быстрый формат
 - ▶ **CSR** — немного медленнее **CSC**, но часто сопоставим по времени
 - ▶ **LIL** — значительно медленнее
 - ▶ **DOK** — также медленнее, чем **CSR/CSC**
- Обоснование:
 - ▶ Арифметические операции
 - ▶ Транспонирование **CSC** даёт **CSR** (транспонирование **CSR** даёт **CSC**), поэтому центральная формула этого алгоритма — **CSR @ front @ CSC** (при исходном **CSC**) — является более сбалансированной по определению перемножения матриц

Результаты по вопросу №2 (1/2)



Результаты по вопросу №2 (2/2)



Вывод по вопросу №2

- При малых размерах (1-5 вершин) разница в производительности между реализациями минимальна и зависит от графа и запроса, но при большем размере стартового множества **tensor_based_rpq** лучше **ms_bfs_based_rpq**

Объяснение результата с помощью профилирования

Данные: граф core с 10% стартовым мн-вом, запрос $(m_1|m_2) * m_3$

- **ms_bfs_based_rpq:** 690453 вызовов функций за 0.452 с
- **tensor_based_rpq:** 317107 вызовов функций за 0.202 с

Анализ **ms_bfs_based_rpq:**

- Проблема №1: Постоянное создание и валидация матриц
 - ▶ `scipy.sparse._compressed.__init__` — 3865 вызовов, 0.174 с
 - ▶ `scipy.sparse._lil.__init__` — 34 создания матриц за 0.083 с
 - ▶ `scipy.sparse._compressed.check_format` — 5403 проверок форматов за 0.056 с
- Проблема №2: Множество мелких матричных операций
 - ▶ 684 операции матричного умножения (`__matmul__`) за 0.131 с
 - ▶ 456 операций сложения матриц за 0.078 с

Анализ **tensor_based_rpq:**

- `networkx.add_edge` — 2794 вызова, 0.067 с
- остальные вызовы — работа с `pyformlang` (`to_networkx`, `from_networkx`, `add_transition`)

- 1 Лучшее представление разреженных матриц:
 - ▶ для `tensor_based_rpq` — **CSR**
 - ▶ для `ms_bfs_based_rpq` — **CSC**
- 2 Для любого размера стартового множества `tensor_based_rpq` быстрее `ms_bfs_based_rpq`

Ссылка на pull request:

<https://github.com/RodionovMaxim05/formal-lang-course/pull/5>